

CS402/MA492 Cryptography

Tobias Rossmann

School of Mathematics, Statistics and Applied Mathematics

NUI Galway

Lecture 23:

Cryptography in Python, part 5

Topics

- Breaking Hill ciphers (A2/Q2).
- Known-plaintext attacks against LFSRs (A2/Q3).
- Maximal least periods of bitstreams generated by LFSRs (A2/Q4).
- Using the `RSA` class of the `cs402` Python module (A3/Q2).
- Exploiting weak RSA keys (A3/Q3).
- Diffie-Hellman key exchange (A3/Q4).

Setup

```
In [1]: from cs402 import *
```

```
In [2]: %matplotlib inline
```

Breaking a Hill cipher

A Hill Cipher over the above 27-symbol alphabet was used to produce the ciphertext in the file [a2q2-cipher1.txt](http://www.maths.nuigalway.ie/~rossmann/cs402/a2q2-cipher1.txt) (<http://www.maths.nuigalway.ie/~rossmann/cs402/a2q2-cipher1.txt>). Use the same cipher with the same key to encrypt the string WCHURCHILL of length 10. Carefully enter the encrypted string as a string of precisely(!) 10 symbols; capitalisation matters.

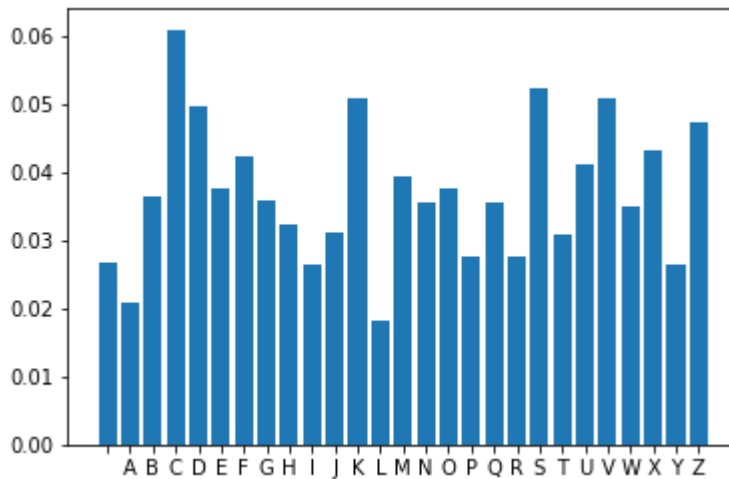
- Note that the dimension of the Hill cipher is not specified.
- Is it possible that a 1-dimensional cipher was used?

Read the ciphertext

```
In [3]: f = open('a2q2-cipher7.txt')
cipher = f.read()
f.close()
```

Frequency analysis

```
In [4]: frequency_histogram(cipher)
```



- This doesn't look like the usual frequency distribution of letters in English at all.
- Perhaps a 2-dimensional Hill cipher was used?
- We should therefore look at *digraph frequencies*.

```
In [5]: digraph_ranking(cipher)
```

Most frequently used digraphs:

1.	"UM"	88
2.	"VK"	85
3.	"BD"	81
4.	"QK"	69
5.	"WN"	59
6.	"XZ"	58
7.	"MZ"	54
8.	"SU"	47
9.	"VX"	44
10.	" P"	43

Note that the multiplicities of the most frequent digraphs are quite close to each other.

- The next step is to make educated guesses about the plaintext digraphs corresponding to some of the digraphs above.
- We first consider a reasonably large piece of text.

```
In [6]: f = open('alice.txt') # plaintext from Lecture 16, different alphabet!
alice = f.read()
f.close()
digraph_ranking(alice)
```

Most frequently used digraphs:

1. "e "	326
2. " t"	296
3. "he"	282
4. "th"	227
5. " a"	213
6. "d "	204
7. "t "	197
8. " s"	195
9. ", "	171
10. "er"	161

Again, some of these digraphs have very similar multiplicities. For short texts, it can therefore be very easy to misidentify digraphs.

We might suspect that "E ", " T" , "HE" , and "TH" are frequent digraphs in the plaintext corresponding to cipher .

After experimenting with various combinations, we arrive at the following candidate:

Plaintext	Ciphertext
E	UM
TH	VK
T	BD

Let's translate this into numbers:

```
In [7]: print(string_to_int_list(ALPHABET27, 'E '))
print(string_to_int_list(ALPHABET27, 'TH'))
print(string_to_int_list(ALPHABET27, ' T'))
```

```
[4, 26]
[19, 7]
[26, 19]
```

```
In [8]: print(string_to_int_list(ALPHABET27, 'UM'))
print(string_to_int_list(ALPHABET27, 'VK'))
print(string_to_int_list(ALPHABET27, 'BD'))
```

```
[20, 12]
[21, 10]
[1, 3]
```

We therefore seek to find a matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

with

$$A \begin{bmatrix} 4 \\ 26 \end{bmatrix} \equiv \begin{bmatrix} 20 \\ 12 \end{bmatrix}, \quad A \begin{bmatrix} 19 \\ 7 \end{bmatrix} \equiv \begin{bmatrix} 21 \\ 10 \end{bmatrix}, \quad A \begin{bmatrix} 26 \\ 19 \end{bmatrix} \equiv \begin{bmatrix} 1 \\ 3 \end{bmatrix},$$

where the congruences are modulo 27.

Recall that

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}.$$

The above congruences give us six congruences in the variables a, b, c, d .

The first two of these congruences combine to

$$\begin{bmatrix} 4 & 26 & 0 & 0 \\ 0 & 0 & 4 & 26 \\ 19 & 7 & 0 & 0 \\ 0 & 0 & 19 & 7 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} \equiv \begin{bmatrix} 20 \\ 12 \\ 21 \\ 10 \end{bmatrix} \pmod{27}$$

We can attempt to solve this using the function `solve_modular_linear_system` from Lecture 12.

```
In [9]: M = [[4,26,0,0],[0,0,4,26],[19,7,0,0],[0,0,19,7]]
        b = [20,12,21,10]
        solve_modular_linear_system(M,b,27)
```

```
Out[9]: [4, 23, 2, 23]
```

So we suspect that the key used was

$$A = \begin{bmatrix} 4 & 23 \\ 2 & 23 \end{bmatrix}.$$

Let's try that:

```
In [ ]: A = [[4,23],[2,23]]
        H = HigherAffineCipher(ALPHABET27,2)
        H.decrypt_string((A,[0,0]), cipher)
```

Your version of this problem may have required different combinations involving the first few digraphs.

Known-plaintext attack against LFSRs

A linear feedback shift register of length 8 with connection polynomial

$C(X) = 1 + aX + bX^2 + cX^3 + dX^4 + eX^5 + fX^6 + gX^7 + hX^8$ is used to generate a pseudo-random binary sequence. This pseudo-random sequence is used as the key stream of a stream cipher. It is known that when the cipher is applied to the plaintext string

$[0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1]$

the corresponding ciphertext string is

$[1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1]$.

Determine the coefficients of the connection polynomial $C(X)$.

We use the method from Lecture 13. Let us first generate the initial segment of the key stream:

```
In [11]: plain = [0,0,0,1,0,0,0,0,1,1,0,0,1,0,1,0,0,1,1,1,0,1,0,1,1,0,1,1,0,1]
cipher = [1,1,1,1,1,0,1,1,0,1,0,1,1,0,0,1,1,1,0,0,0,1,0,1,0,0,0,0,1]
keys = []
for i in range(len(plain)):
    keys.append((plain[i] + cipher[i]) % 2)
```

```
In [12]: # Alternatively, for people who appreciate Python one-liners:
keys = [(a + b) % 2 for a,b in zip(plain,cipher)]
```

We can now write down the matrix A and vector b on p.54 of the lecture notes. We'll use some Python magic (slices and list comprehension) to make our life easier here; this can also be done by hand, of course.

```
In [13]: A = [ keys[i:i+8][::-1] for i in range(8)]
print(A)
b = keys[8:16]
print(b)

[[1, 1, 0, 1, 0, 1, 1, 1], [1, 1, 1, 0, 1, 0, 1, 1], [0, 1, 1, 1, 0, 1, 0, 1], [0, 0, 1, 1, 1, 0, 1, 0], [1, 0, 0, 1, 1, 1, 0, 1], [0, 1, 0, 0, 1, 1, 1, 0], [0, 0, 1, 0, 0, 1, 1, 1], [1, 0, 0, 1, 0, 0, 1, 1]]
[1, 0, 0, 1, 0, 0, 1, 1]
```

```
In [14]: solve_modular_linear_system(A,b,2)
```

```
Out[14]: [0, 1, 0, 1, 1, 1, 1, 1]
```

Hence $b = d = e = f = g = h = 1$ and all other coefficients are zero.

Let's verify this:

```
In [15]: L = LFSR(8, [2,4,5,6,7,8])
S = StreamCipher(L)
S.encrypt_bit_string(keys[:8], plain) == cipher
```

```
Out[15]: True
```

Maximal least periods

A linear feedback shift register of length L is represented by its connection polynomial $C(X) = 1 + c_1 X + c_2 X^2 + \dots + c_L X^L$. It can be shown that if $c_L = 1$ then the LFSR generates a purely periodic pseudo-random sequence for any initial state $[s_0, s_1, \dots, s_{L-1}]$. The size of the minimal period can depend on the initial state. What is the largest possible minimal period of a pseudo-random sequence generated by the LFSR of length 8 with connection polynomial $C(X) = 1 + X^7 + X^8$?

We could just use nested `for` loops as in Lecture 12. A cleaner way is to combine `product` from the `itertools` module in the Python standard library, the builtin `max` function, and generator comprehension:

```
In [16]: from itertools import product
L = LFSR(8, [7,8])
max(L.period(x) for x in product([0,1], repeat=8))
```

```
Out[16]: 63
```

Using the RSA class of the `cs402` module (A3/Q2).

Alice and Bob are planning to conduct a secret meeting without Oscar. By spying on a conversation that Alice and Bob had at a coffee shop, Oscar learned that the meeting will take place next Saturday. However, Alice refused to discuss the location of the meeting in public. Instead, she used Bob's public RSA key to send him the following encrypted message:

6Fs0Jh17XQk wcg3kZUIn.4SApn

Bob's public key

$$(n, e) = (7231645985673347207280720222548553948759779729581, 3)$$

is stored on his web page. Oscar managed to steal Bob's laptop which contained the private key (n, d) , where

$$d = 4821097323782215625692549251331855329314609896043$$

Where will Alice and Bob meet?

```
In [17]: n,e,d = 7231645985673347207280720222548553948759779729581, 3, 4821097323782
215625692549251331855329314609896043
R = RSA(ALPHABET64)
cipher = "6Fs0Jh17XQk wcg3kZUIn.4SApn"
R.decrypt_string((n,d), cipher)
```

```
Out[17]: 'Bangor Erris'
```

Exploiting weak RSA keys (A3/Q3).

Alice and Bob met, only to find that Oscar was already waiting for them. They decided to meet again the following week. This time, Bob uses Alice's public RSA key

$$(n, e) = (15241604814814604814814604814814609737853, 47)$$

to send her the encrypted message 4MKBJEzknCpaWSaMqQ82PH . Over the coming days, despite repeated attempts, Oscar fails to steal Alice's laptop. However, he had previously managed to sabotage the key generation step of the RSA implementation that she used to make it less secure. Where will Alice and Bob meet the following week?

```
In [18]: n,e = 15241604814814604814814604814814609737853, 47
cipher = "4MKBJEzknCpaWSaMqQ82PH"
```

Let's try to factor $n = pq$. We can then repeat the part of the key generation step that produces Alice's private key (n, d) .

We'll just attempt trial division. If n has a very small prime divisor, this should be feasible.

```
In [19]: from math import floor, sqrt
        for d in range(2, floor(sqrt(n)) + 1):
            if n % d == 0:
                print('Found divisor:', d)
                break
```

Found divisor: 123457

```
In [20]: p = 123457
        q = n // d
        print(n == p*q)
        q
```

True

Out[20]: 123456789123456789123456789123456829

```
In [21]: d = modular_inverse(e, (p-1)*(q-1))
        d
```

Out[21]: 1297147349619189902810679264512807332559

```
In [22]: R.decrypt_string( (n,d), cipher)
```

Out[22]: 'Shillelagh'

Diffie-Hellman key exchange (A3/Q4).

Alice and Bob choose the abelian group $\mathbb{Z}_{1019}^\times$ and generator $g = 117$ to perform a Diffie-Hellman key exchange. Alice secretly chooses $a = 14$ and Bob secretly chooses $b = 13$. What is the numerical value of their shared key?

Alice and Bob choose the abelian group $\mathbb{Z}_{1019}^\times$ and generator $g = 117$ to perform a Diffie-Hellman key exchange. Alice secretly chooses $a = 14$ and Bob secretly chooses $b = 13$. What is the numerical value of their shared key?

```
In [23]: pow(117, 14*13, 1019)
```

Out[23]: 921